

Do all the references actually exist? If they do not, this is typically seen as a proof that we had this paper AI generated. It will then likely be desk-rejected.
I could not find:
- Abhimanyu Das et al. Time series foundation models and forecasting
This one seems to have a slightly different name:
Ya Su et al. Robust anomaly detection for multivariate time series through stochastic recurrent neural network (omnianomaly).
This seems to have the wrong author:
Gerald Woo et al. Chronos: Learning the language of time series. arXiv preprint, 2024a.
Two more points:
A) When citing arxiv papers it is usually so that one puts the identifier in the citation, such as arXiv:2201.12740. B) Sometimes capitalization is important. The references currently write everything in lowercase, but some abbreviations are chosen by their authors to capitalize some words. It is usually nice to follow their choices.

FactoryBench: Benchmarking Machine Understanding via Question-Answering

Yanis Merzouki
ETH Zurich
ymerzouki@student.ethz.ch

Coral Izquierdo Muñiz
UC3M
coral.izquierdo@alumnos.uc3m.es

Matei Ignuta-Ciuncanu
Imperial College London
mic18@imperial.ac.uk

Philipp Petersen
University of Vienna
philipp.petersen@univie.ac.at

Jonas Petersen
Forgis AG, ETH Zurich
jonas.petersen@forgis.com

Abstract

Time-series models are widely used in industrial monitoring tasks such as forecasting, anomaly detection, and signal analysis. While highly effective for these objectives, they are often opaque and limited in their ability to provide structured reasoning for engineering decision-making. Large language models (LLMs), in contrast, can generate coherent, context-aware explanations and support multi-step reasoning. However, general-purpose LLMs still struggle when applied directly to dense multivariate sensor streams and machine-specific diagnostics without adaptation or external tools, and even then they often tend to learn wrong signals even when answering correctly to a question. This raises a central question: how can we verify that these systems truly understand machine behavior rather than only generating plausible output?

We introduce **FactoryBench**, a benchmark for evaluating time series models and LLMs on machine understanding over industrial time-series data. We propose a scalable Q&A generation framework built around structured question templates, present **FactoryWave** (a dense, multitask and multivariate sensor dataset from real robotic systems) and construct FactoryBench as a large-scale benchmark grounded in FactoryWave alongside open-source datasets and simulations. Together, these

provide a rigorous testbed for evaluating reasoning, causal understanding, and decision support over industrial signals.

1 Introduction

Modern industrial systems generate large volumes of multivariate time-series data from sensors, actuators, and controllers. In robotic manufacturing settings, these signals encode joint states, torques, forces, velocities, contact events, task phases, and fault indicators. Extracting actionable knowledge from such data is central to monitoring, diagnostics, anomaly detection, and decision support.

Traditional time-series models excel at narrow tasks such as prediction, classification, and anomaly detection Zhou et al. [2021], Su et al. [2019], Audibert et al. [2020]. However, these models are often specialized and difficult to interpret. They typically output labels or numeric predictions without explicit reasoning or higher-level explanations, and generalize poorly outside of the specific task they are trained on Chalapathy and Chawla [2019], Lavin and Ahmad [2015], Zeng et al. [2023], which limits their direct utility for automated engineering decision-making.

Large language models, in contrast, exhibit strong general reasoning abilities and can produce structured explanations in technical contexts Wei et al. [2022], Yao et al. [2023b]. They can integrate textual information, follow logical chains, and generate interpretable outputs. Yet, when applied directly to dense numerical time series, general LLMs still underperform without adaptation Gruver et al. [2023], Jin et al. [2024]. In contrast, specialized transformer-based architectures have become increasingly effective for time-series forecasting and representation learning Vaswani et al. [2017], Zhou et al. [2021], Wu et al. [2021], Zhou et al. [2022], Nie et al. [2023], Ansari et al. [2024], Das et al. [2024].

A promising paradigm is to build LLM agents equipped with specialized tools, including time-series models and signal-processing modules Schick et al. [2023], Yao et al. [2023a], Qin et al. [2023]. These agents can combine language reasoning with domain-specific computation. Nevertheless, evaluating whether such systems truly reason about machine behavior rather than pattern-matching on surface textual cues remains challenging. By machine behavior understanding we mean the ability to:

- I. interpret the current operational state from raw multivariate signals
- II. predict the effect of an intervention on the system
- III. reason counterfactually about what would have happened under alternative histories
- IV. recommend remedial actions grounded in physical and engineering constraints

Current LLMs typically fail on II, III and IV because they lack access to calibrated dynamics and cannot reliably ingest long, dense numerical streams. Existing benchmarks mostly target textual reasoning, code generation, or generic multimodal understanding Hendrycks et al. [2021], Srivastava et al. [2023], Yue et al. [2024], leaving a gap in rigorous evaluation of machine-behavior reasoning over industrial telemetry.

To address this gap, we propose FactoryBench, a benchmark designed to evaluate machine-behavior reasoning in time series models and LLMs. FactoryBench is grounded in FactoryWave, a dense multivariate time-series dataset collected from one-armed robotic systems spanning both collaborative and industrial manufacturing platforms, with synchronized setpoint, context, feedback, and effort signals recorded at 83 and 125 Hz (KUKA KR10 and UR3e respectively). Industrial-robot data of this kind is rare in public benchmarks: these systems are almost exclusively deployed in production environments with safety enclosures that restrict access, proprietary controllers that expose limited telemetry, and recordings that typically fall under plant-level confidentiality agreements. As a result, existing robotic datasets are dominated by cobot platforms, and to our knowledge FactoryWave is the first extensive anomaly dataset to (partly) cover an industrial robot, explicitly aimed at bridging this cobot-to-industrial-machine gap.

We also introduce a scalable question-generation framework composed of 20 structured templates spanning a wide range of reasoning types and applicable to general machine data flows. These templates enable systematic construction of question-answering tasks that probe state understanding, intervention reasoning, counterfactual analysis, and decision-making in machine contexts. Using

this framework and the FactoryWave dataset, we build FactoryBench, a large-scale Q&A dataset specifically designed to evaluate time-series understanding and engineering reasoning in LLM agents.

By bridging the gap between general language reasoning and specialized time-series modeling, FactoryBench provides a principled foundation for studying machine-behavior reasoning in industrial environments.

2 Related work

2.1 LLM agents and tool use

Recent advances in LLM reasoning include prompting and deliberative strategies (e.g., Chain-of-Thought and Tree-of-Thought) that improve multi-step performance Wei et al. [2022], Yao et al. [2023b], alongside tool-augmented paradigms such as Toolformer and ReAct that enable grounded computation through external modules Schick et al. [2023], Yao et al. [2023a], Qin et al. [2023]. These ideas motivate industrial agent design where symbolic reasoning must be combined with numerical workflows.

2.2 Time-series learning for industrial signals

Time-series modeling has advanced through transformer-based architectures (Informer, Autoformer, FEDformer, PatchTST) Zhou et al. [2021], Wu et al. [2021], Zhou et al. [2022], Nie et al. [2023], strong alternative baselines such as TFT and N-BEATS Lim et al. [2021], Oreshkin et al. [2020], and critical comparisons against simpler linear models Zeng et al. [2023]. Foundation-model directions (e.g., Chronos) further explore transfer and zero/few-shot adaptation Ansari et al. [2024], Das et al. [2024]. In parallel, anomaly and reliability monitoring literature includes OmniAnomaly, USAD, and Deep SVDD Su et al. [2019], Audibert et al. [2020], Ruff et al. [2018], with surveys and benchmarks highlighting persistent gaps in interpretability and root-cause actionability Chalapathy and Chawla [2019], Lavin and Ahmad [2015].

2.3 Causality and benchmark gaps

Causal frameworks provide formal tools for interventions and counterfactuals Pearl [2009], Peters et al. [2017], Rubin [1974], while temporal methods such as Granger causality and modern nonlinear causal discovery support directional reasoning in time series Granger [1969], Runge et al. [2019]. Existing broad benchmarks (MMLU, BIG-bench, MMMU) Hendrycks et al. [2021], Srivastava et al. [2023], Yue et al. [2024] and classical time-series resources Laptev et al. [2015], Dau et al. [2019], Wen et al. [2022] do not directly evaluate machine-centered reasoning over industrial telemetry. FactoryBench targets this gap by jointly testing temporal interpretation, causal reasoning, and engineering decision support. Table 1 positions FactoryBench against closely related Q&A and time-series benchmarks along eight axes. The column labels refer to concepts introduced later in the paper: Counterfactual denotes questions that reason about alternative histories (Level 3, Section 3.1); Free-Form denotes open-ended natural-language answers evaluated via an LLM-as-judge protocol (Section 3.2); and Novel Dense TS indicates whether the benchmark contributes a new, high-frequency multivariate dataset rather than relying exclusively on public sources.

3 FactoryBench framework

Figure 1 provides a high-level overview of the FactoryBench framework. The left panel illustrates our four-level causal evaluation hierarchy from state interpretation to decision-making together with the setpoint–context–effort (SCE) schema that underlies the universal fault definition introduced in Section 3.3. The right panel summarizes the scalable question-generation pipeline, which expands a small set of hand-authored templates into a large, semantically diverse Q&A pool, and contrasts the resulting benchmark against existing time-series and robotics Q&A datasets along the axes introduced in Table 1.

Table 1: Comparison of FactoryBench with existing benchmarks. “Counterfactual” = questions reasoning over alternative histories; “Free-Form” = open-ended answers judged by LLMs; “Novel Dense TS” = contributes a new multivariate high-frequency dataset.

Benchmark	Machine/Robot	Multivariate TS	Size	Counterfactual	Ranking	Numerical	Free-Form	Novel Dense TS
TimeSeriesExam	×	×	~700	×	✓	×	×	×
ChatTS	×	✓	~134k	×	✓	✓	×	×
EngineMT-QA	✓	✓	~110k	×	✓	✓	✓	×
TSAQA	Partial	×	~210k	×	✓	×	×	×
Time-MQA	×	✓	~200k	×	✓	✓	✓	×
MTBench	×	✓	?	×	✓	✓	✓	×
QuAnTS	×	✓	~150k	×	✓	✓	✓	×
CLEVR	×	×	~1M	×	×	✓	×	×
CLEVRER	×	×	~305k	✓	×	✓	✓	×
FactoryBench (Ours)	✓	✓	TBD	✓	✓	✓	✓	✓

Table 2: FactoryBench levels, what they test, example questions, and expected answer formats.

Level	Example Question	Task	What It Tests	Answer Format
1	“We want to isolate the lifting phase in the robot’s time series. Assuming a fixed window length of 15 timesteps, at which timestamp should the window begin?”	State	Can the model understand what the machine is doing and how it is behaving?	Tensor
2	“A collision with a foam cube occurs at T=850ms. Rank the following signal segments (A–D) in the order you would expect them to appear after the event.”	Intervention	Can the model reason about how the machine will behave in the face of an event?	Ranking (permutation)
3	“Had a payload misconfiguration occurred at T=200ms, what would be the expected value of the target torque on joint 2 at T+50ms in this counterfactual case?”	Counterfactual	Can the model reason accurately about theoretical scenarios?	Scalar (float)
4	“Given the sensor stream below, does the machine show signs of anomalous behavior? If yes, identify the most likely root cause and the steps to fix it.”	Decision	Can the model make informed decisions about the machine?	Free-form text

3.1 Four levels of machine understanding

Our four-tier hierarchy is explicitly built on top of Pearl’s ladder of causation association, intervention, and counterfactual reasoning which has become the organizing principle for causal inference and is increasingly adopted as an evaluation axis for machine-learning systems Pearl [2009], Peters et al. [2017], Jin et al. [2023], Pawlowski et al. [2020], Peyrard and West [2020]. We extend the three causal rungs with a fourth decision-making rung that reflects how industrial robotic platforms are actually operated: after interpreting state and causal relationships, operators must select and execute corrective procedures, typically prescribed by vendor manuals (e.g., Universal Robots error-code handbooks). This final rung aligns with the diagnostic-then-act loop that is standard in fault-tolerant control. Grounding the hierarchy in these established principles ensures that each level probes a qualitatively distinct reasoning skill and that failure modes are interpretable in terms of the underlying causal or decision-theoretic primitive.

To systematically evaluate machine-behavior reasoning, we organize question-answering tasks according to this four-tier hierarchy, each probing distinct reasoning capabilities:

- **Level 1: State.** This level assesses the agent’s ability to interpret the current state of the machine, including detection of anomalies, identification of operational modes, and

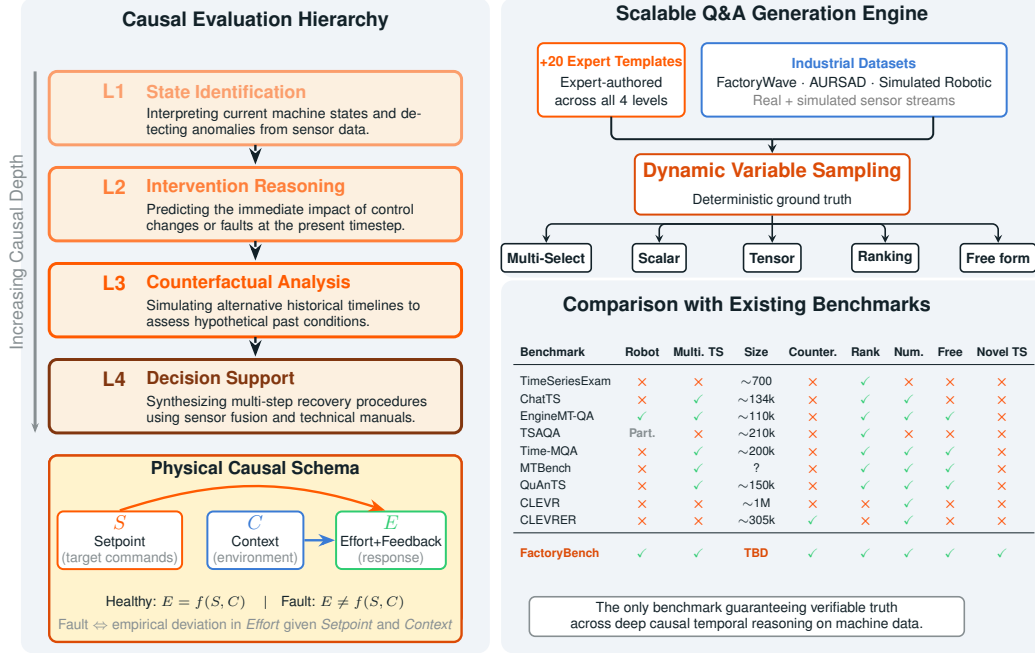


Figure 1: Overview of the FactoryBench framework. The left panel shows the four-level causal evaluation hierarchy and the physical causal schema. The right panel details the scalable Q&A generation engine and compares FactoryBench against existing benchmarks.

The benchmark comparison table in the right panel is repeated elsewhere in the paper and should be removed. The two graphs on the left are talking about different concepts and should be separated

recognition of sensor patterns. Questions at this level require accurate extraction and interpretation of time series features.

- **Level 2: Intervention.** At this level, the agent must reason about the consequences of interventions or events occurring at the present timestep that might perturb the distribution of machine states. Tasks include predicting the immediate impact of control actions, diagnosing faults as they arise, and understanding causal relationships in real time.
- **Level 3: Counterfactual.** This level evaluates the agent’s ability to reason about hypothetical scenarios, such as the effect of an event or intervention at a previous timestep. Questions require the agent to simulate alternative histories and assess how outcomes would differ under counterfactual conditions, while still considering the history they know.
- **Level 4: Decision Making.** The highest level encompasses complex decision-making tasks, where the agent must generate a sequence of actions or recommendations based on the time series and a prompt. This includes troubleshooting, optimization, and planning, requiring integration of state interpretation, causal reasoning, and goal-directed synthesis.

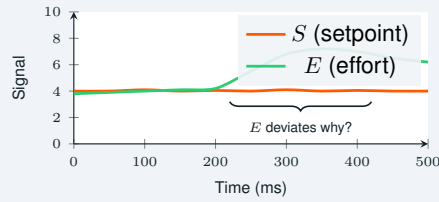
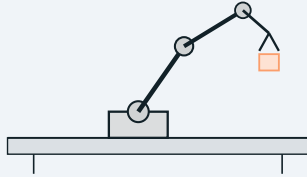
By structuring Q&A tasks along these four levels, FactoryBench enables rigorous and granular assessment of machine-behavior reasoning, from basic state recognition to advanced decision support. Figure 2 instantiates Pearl’s three causal rungs (L1–L3) on a concrete robotic arm; FactoryBench’s L4 then layers decision-making on top of these rungs.

3.2 Answer formats

FactoryBench uses four answer formats, chosen to balance evaluation rigor with scalability. Each format supports deterministic scoring (with the exception of free form), and has been chosen in order to make questions very easily checkable, but also hard to answer in the absence of correct reasoning.

L1 · Seeing (Association) - What is?

$$P(E | S, C)$$

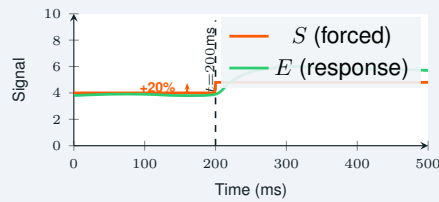
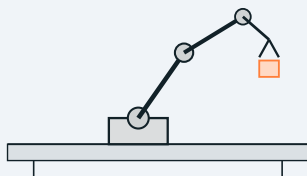


"I observe that when the payload is heavier, motor current is higher."

Correlation from passive observation no action taken.

L2 · Doing (Intervention) - What if I do X?

$$P(E | \text{do}(S))$$

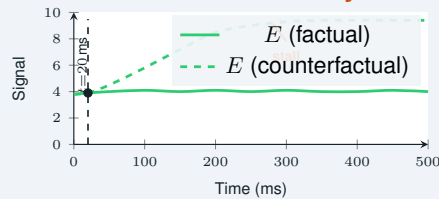
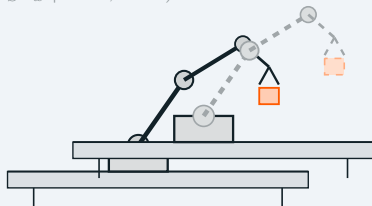


"If I command +20% torque now, the motor current will rise to ~X A."

Predicting the effect of an action at the present timestep.

L3 · Imagining (Counterfactual) - What if I had acted differently?

$$P(E_{S=x} | S=s, E=e)$$



"Given what actually happened, if torque had been raised to X at t = 20ms instead, the arm would have stalled here."

Reasoning over alternative histories, conditioned on what we observed.

Figure 2: Pearl's three levels of causal reasoning instantiated for robotic machine-behavior. **L1 (Seeing/Association)** captures correlations from passive observation. **L2 (Doing/Intervention)** predicts the effect of an active intervention at the present timestep. **L3 (Imagining/Counterfactual)** reasons over alternative histories conditioned on the observed trajectory. FactoryBench extends Pearl's three causal rungs with a fourth decision-making level (L4) that maps observed symptoms to remedial procedures.

Drawing of robotic arm is cool but useless: it doesn't really represent the level being mentioned. Also Level 4 is missing, which is a weird choice. Furthermore, decrease size since we are way over the page limit.

- **Multi-Select MC.** The model is presented with four independent statements (A–D) about the outcome of an event or intervention, and must select any subset of statements. Evaluation gives a score of 1 for exact match, 0.5 for 1 mistake, and 0 otherwise.
- **Ranking.** The model is given four time-series segments or outcomes (A–D) and must order them according to a specified criterion (e.g., severity of deviation, magnitude of a signal). The answer is a permutation string (e.g., DABC). Evaluation uses exact-match rate and Kendall’s τ rank correlation against the ground-truth ordering.
- **Tensor.** The model must output at least one specific scalar value such as the expected sensor reading following an intervention with no multiple-choice scaffolding. The answer is a list of floating-point numbers (possibly one). Evaluation is done on each scalar separately, and uses mean absolute percentage error (MAPE) and a threshold-based accuracy metric (prediction within $\pm k\%$ of ground truth), with partial points given to all correct scalars given.
- **Free-Form.** The model produces an open-ended natural language response such as a diagnosis, recommended action sequence, or causal explanation. Because no single deterministic answer exists, evaluation is performed via an LLM-as-judge voting protocol: three independent frontier LLMs each compare the model’s response against the ground-truth reference and cast a verdict of **Wrong** (0), **Neutral** (0.5), or **Correct** (1). The final score is the majority vote across the three judges, with ties broken by averaging.

3.3 Time series data and causal schema (SCE)

To ensure that the dataset structure itself encodes causality, FactoryBench organizes all time-series signals into three causal groups, answering the core question: “*What was the machine told to do, and what did it actually do?*”

- **Setpoint:** The controller’s command target position, velocity, and acceleration per axis.
- **Context:** Physical conditions affecting behavior payload, temperature, material properties. Split into *static* (episode metadata) and *dynamic* (time-series columns).
- **Effort + Feedback:** The machine’s response motor current (effort), actual position (feedback), vibration, and acoustic emission.

This structure enables a universal fault definition. Under healthy operation, the measured Effort+Feedback response $\mathbf{y}_t$ is defined as a direct function of the Setpoint command $\mathbf{S}$ and the Context $\mathbf{C}_t$:

$$\mathbf{y}_t = f(\mathbf{S}, \mathbf{C}_t)$$

where f represents the nominal plant dynamics (inverse-dynamics-plus-controller transfer) calibrated per robot. Faults manifest as structured deviations from this expected baseline specifically, residuals $\mathbf{y}_t - f(\mathbf{S}, \mathbf{C}_t)$ that form a clear, fault-specific spatiotemporal pattern. Because FactoryBench supplies the paired $(\mathbf{S}, \mathbf{C}_t, \mathbf{y}_t)$ streams, this residual can be computed directly, establishing a single operational definition of a fault that applies across machines and tasks.

This data is sourced from:

- **FactoryWave:** A custom dataset generated from one-arm robotic platforms executing canonical industrial tasks (e.g., pick-and-place) under varied conditions and systematically injected anomalies.
- **Open Source Datasets:** Adapted datasets such as Aursad and Voraused, offering diverse industrial scenarios and preprocessed to conform to the unified episode structure.
- **Simulations:** Synthetic time-series data generated from simulated robotic systems to enable controlled experimentation, ablation studies, and validation across both physical and virtual domains.

The two open-source datasets integrated into FactoryBench are summarized in Table 3; both provide the full setpoint–context–effort triplet at 100 Hz or higher and have been relabelled to conform to our unified episode schema.

Consider removing simulations from this list since we’re only using that dataset for sim2real analysis.

Table 3: Open-source datasets incorporated into FactoryBench.

Dataset	Robot	Size	Frequency	Task	# Anomaly Types
AURSAD	UR3e	?	100 Hz	Screwing	4
voraus-AD	UR5	?	100/500 Hz	Pick and Place	12

4 Reliable Q&A generation (key contribution)

4.1 At scale generation via extensive labelling

Scalable ground-truth generation is the central challenge of any Q&A benchmark grounded in raw data. FactoryBench addresses this by coupling a structured labelling ontology with a context-free grammar (CFG)-style template system, designed by PhD-level experts in robotics. Rather than annotating individual questions by hand, each template is parameterized: concrete values are filled at generation time from the episode data and its associated labels. The context time series used to fill the variables of the question is sampled uniformly by datasource (sim vs open source vs FactoryWave), dataset (if looking at open source), experiment, difficulty, length and then placement in that order. This yields a combinatorial expansion from a small set of carefully designed templates into a large, diverse question pool.

Guarding against template shortcuts. A natural concern is that template-generated benchmarks can be solved by learning the surface structure of templates rather than the underlying reasoning. Our primary defense is structural: each template admits a combinatorially large instantiation space over signal names, timestamps, prediction horizons, payload conditions, and injected-fault types, so two questions sharing the same template rarely share more than a fraction of their variable slots. Crucially, the correct answer is never recoverable from the question text alone it is determined by the paired time series, which varies across instantiations.

Variable sampling. Each question template contains variable slots that are filled at generation time from one of two sources: (i) label information read directly from the episode data (e.g., signal names, timestamps, anomaly labels, root causes), or (ii) a predefined sampling distribution attached to that variable (e.g., prediction horizons, numerical thresholds, or curated vocabularies for discrete slots). The discrete vocabularies are seeded from the label sets of AURSAD and voraus-AD and extended to cover cases specific to FactoryWave; they are released in full so every question instantiation is inspectable and reproducible.

Template design. Each of the 20 question templates was manually authored to probe one specific machine understanding level, while remaining general enough to admit a wide range of concrete instantiations. Templates are parameterized over episode segments, signal names, event descriptions, timestamps, and predicted values. Answer options for single and multi-select questions are drawn from a shared pool of predefined verifiable statements, each paired with a rule that can be evaluated deterministically against the time series, given the densely labelled data. This design ensures that ground truth is never imputed or inferred it is computed directly from labeled episode data making the benchmark both reliable and fully reproducible.

[TODO:
check the
exact number
when level 4
implemented]

4.2 Density of FactoryWave

FactoryWave is collected from two physical robots (UR3e and the KUKA KR10) executing up to three canonical industrial tasks each: pick-and-place, screwing (UR3e only), and peg-in-hole. Each complete task cycle constitutes one episode, recorded at 500 Hz across sensor channels covering joint setpoints, feedback, torques, speeds, estimated contact forces, TCP pose, gripper state, and task-phase labels.

find number

Episodes are organized into four experiment types. **Normal** episodes capture nominal operation across three payload levels (light, medium, heavy for pick-and-place). **Trajectory optimization** episodes run the same task with randomly varied waypoint sequences, recording episode-level duration and energy metadata to support Level 4 ranking questions. **Fault** episodes inject one of 37 fault types (spanning gripper failures, payload misconfigurations, software faults, collisions, and peg-in-hole-specific anomalies), each recorded with fault-specific metadata and, where applicable, a

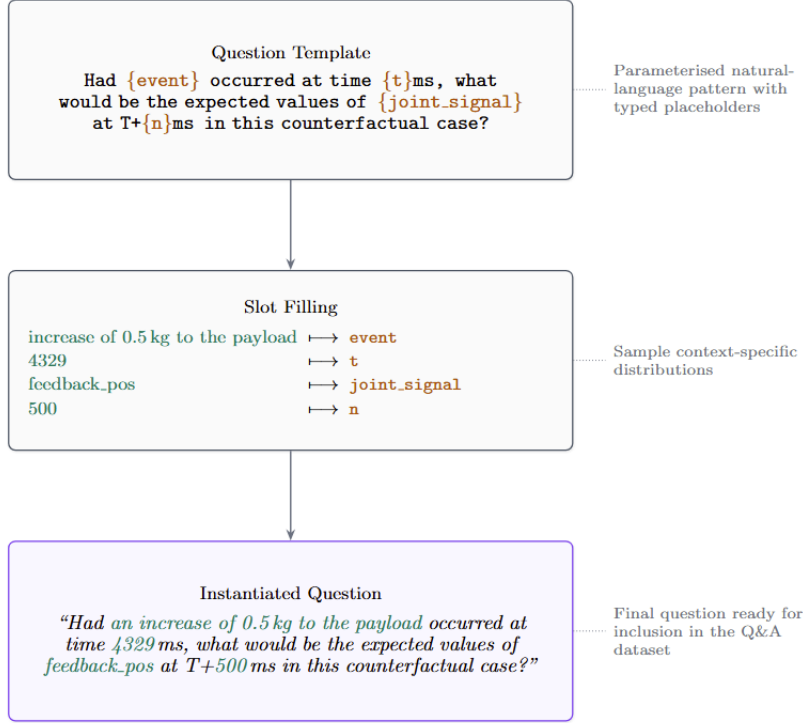


Figure 1. Chain structure for generation of unique synthetic questions at scale. A parameterised template (Step 1) defines the question skeleton with typed placeholder slots. Slot filling (Step 2) binds each placeholder to a concrete value sampled from the scenario context. The resulting instantiated question (Step 3) is ready for inclusion in the Q&A evaluation dataset.

Figure 3: Question generation pipeline.

precise injection timestep. **Counterfactual** episodes pair a clean baseline run with a fault run whose pre-injection segment most closely matches the baseline (selected by minimum KL divergence), providing ground truth for Level 3 causal reasoning.

Table to summarize all number of episodes and categories.

4.3 Knowledge graph and protocol extraction

Reliable ground truth for Level 4 troubleshooting questions requires more than episode labels: it demands machine-specific recovery protocols grounded in manufacturer documentation. To produce this at scale, we implement a structured parsing pipeline over robot technical manuals.

Error-to-protocol extraction. For each robot in FactoryBench, we parse the manufacturer’s runtime error documentation to extract a structured mapping from error codes to error names, descriptions, and recommended recovery steps. Each entry captures what the controller reports when a fault occurs and what an operator should do to resolve it.

Anomaly-to-error mapping. We then map every anomaly type studied in FactoryBench to the most likely corresponding manufacturer error code. This mapping is established by PhD-level robotics experts who reason about the physical cause of each anomaly and identify which runtime error it would most plausibly trigger on the target robot. The result is a fault-indexed lookup table linking each root cause to a specific error code and its associated protocol.

Expert-authored gap-filling. Not all anomalies correspond to a documented runtime error. Physical fault injections (such as gripper misactivation, peg misalignment, or external collisions) may not produce a well-defined controller error at all. For these cases, robotics experts author recovery protocols directly, following the same structure as the manufacturer documentation to maintain consistency.

Knowledge graph integration. The complete mapping (error-derived and expert-authored protocols alike) is ingested into a knowledge graph alongside robot specifications, task definitions, and signal schema. At question generation time, the pipeline queries this graph to automatically assemble the relevant protocol context and inject it as ground truth into Level 4 troubleshooting Q&A pairs, without requiring per-question human review.

4.4 FactoryBench-Lite: a representative 10% subset

FactoryBench-Lite not yet implemented.

IDEA:

1. run full dataset
2. implement stratified selection preserving per-stratum mean/std of reference-model scores (GHOSTS-style, Frieder et al. [2023])
3. validate via Spearman/Kendall rank correlation and per-level MAE Polo et al. [2024], Bean et al. [2025]

4.5 Pipeline overview

Some practical questions: How can people employ the benchmark? Where can it be found. Will it be maintained and expanded? Will incorrect labels be corrected? Is there a private dataset too? Are parts of the dataset designated for training and parts for testing?

FactoryBench and FactoryWave are released under a license and distributed via a public Hugging Face repository. The release includes the episode data, the generator source code, the full set of 20 question templates with their paraphrase banks, all LLM-as-judge prompts, and both the full benchmark and the FactoryBench-Lite subset (Section 4.4). We provide a versioned release track (e.g., v1.0, v1.1) so that reported numbers always refer to a fixed benchmark state; corrections to labels or templates are published as minor-version updates with a public changelog, and the exact version used in any evaluation is stamped into every result file.

check license

Check the number of question templates when level 4 finished

5 Experiments

5.1 Models to evaluate

We evaluate a cross-vendor panel of frontier LLMs covering both closed-weight and open-weight releases available at submission time (April 2026): on the closed-weight side, GPT-5-mini OpenAI [2025] and Claude Haiku 4.5 Anthropic [2025]; on the open-weight side, Qwen-3.5 Qwen Team, Alibaba [2026], DeepSeek V3.1 DeepSeek-AI [2025], and Mistral-Large-3 Mistral AI [2025]. Alongside these general-purpose models we also run two specialized time-series foundation models, Chronos-2 Ansari et al. [2024] and Moirai Woo et al. [2024], restricted to the forecasting-style subset of FactoryBench whose ground truth is a future scalar or segment of the time series (Tensor-type questions); since these models do not ingest natural-language prompts they are not applicable to the MC, Multi-Select, Ranking, or Free-Form subsets. Finally, we report two baselines for calibration: a random-choice baseline and a simple rule-based threshold detector.

5.2 Experimental configurations

Beyond out-of-the-box evaluation, we investigate how different interventions improve performance:

- **Finetuning:** We finetune open-source models on the FactoryBench Q&A dataset to test learnability of machine understanding, as well as generalizability beyond cobot data.

Table 4: Expected performance across levels (accuracy %).

Level	Random	Current LLMs	Target (w/ prior)	Human Expert
1	50%	85–95%	98%	99%
2	20%	60–75%	85%	95%
3	10%	35–50%	70%	90%
4	5%	15–30%	45%	80%

5.3 Sim2real gap analysis on counterfactual pick-and-place

Counterfactual questions (Level 3) require comparing what *did* happen in an episode against what *would have* happened under a different intervention, holding the rest of the world fixed. In a physical lab this ground truth is impossible to obtain exactly: two real executions never share identical initial conditions, micro-disturbances, or stochastic contact dynamics, so any real-world “counterfactual pair” is only an approximation of the intended do-operation. Simulation removes this obstacle: the simulator state can be deterministically forked at an arbitrary timestep, one branch continues under the baseline policy, the other under the intervention, and the two trajectories share a bit-identical pre-intervention history. This makes simulation the only setting in which Level 3 ground truth can be produced without approximation error. However, simulation gains exact counterfactual fidelity but loses realism: contact dynamics, sensor noise, actuator latency, and manufacturing variability all deviate from the physical system. A model that learns to exploit simulator-specific artefacts may score well on simulated counterfactuals without genuinely understanding the underlying robotic behaviour. We therefore treat the simulated and physical counterfactuals as complementary rather than interchangeable.

FactoryWave counterfactual pipeline (physical). For each selected fault whose injection moment can be precisely timed, we: (i) fix initial conditions (object identity, pose, payload, controller config), (ii) record one clean *baseline* run of the full episode, (iii) record 3–5 *fault* runs under the same initial conditions with the fault event injected at a randomly sampled but fixed timestep, (iv) score each fault run against the baseline on the pre-injection segment using KL divergence, and (v) retain the run with the lowest KL divergence as the counterfactual partner for the baseline. All other candidate runs are discarded. The pair is stored with `cf_fault_id`, `cf_injection_timestep`, `cf_injection_time_s`, and `cf_baseline_episode_id`. Full protocol details are in Appendix C.

IsaacSim counterfactual pipeline (simulation). We reproduce the pick-and-place task in NVIDIA IsaacSim with a URDF-matched UR3 and a matching gripper, workspace layout, and fault applicators. For each simulated episode we run the baseline to completion, then re-run from a snapshot of the simulator state taken at the injection timestep with the fault applicator enabled. Because both branches are initialised from the identical saved state, the pre-intervention trajectories are bit-identical by construction; no KL-based selection or discarding is required. The simulation pipeline can therefore generate arbitrarily many counterfactual pairs at negligible marginal cost, including for faults that are too disruptive or unsafe to stage repeatedly on the physical robot.

Sim2real gap experiment.

5.4 Results

6 Discussion

6.1 Why this benchmark matters

For researchers: First rigorous evaluation of machine understanding across reasoning levels.

For practitioners: Quantifies which AI capabilities are ready for deployment as AI engineers.

For the field: Defines “machine understanding” operationally via Q&A.

6.2 Limitations

- Single machine family (by design, for depth)

Fill in concrete numbers, plots, and per-template gap tables once the IsaacSim pipeline is frozen; see Appendix E.

Update values when final results are computed.

- Simulation-to-real gap in synthetic episodes
- Reliance on LLMs-as-Judge for evaluation of Free-Form questions

7 Conclusion

FactoryBench introduces a systematic approach to evaluating machine understanding via Q&A. By focusing deeply on one machine family and providing reliable answer generation at scale, we enable rigorous comparison of methods across four levels of reasoning from basic state identification to procedure generation grounded in technical manuals.

Our physical validation protocol closes the loop from benchmark to reality, addressing the fundamental question: can AI systems that excel at industrial Q&A actually help in the real world?

References

- Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Sundar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiegner, Danielle C. Maddix, Hao Wang, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. Chronos: Learning the language of time series. *TMLR*, 2024. arXiv:2403.07815.
- Anthropic. Claude Haiku 4.5: Model card, 2025. Anthropic model card.
- Julien Audibert et al. Usad: Unsupervised anomaly detection on multivariate time series. *KDD*, 2020.
- Andrew M. Bean, Nabeel Seedat, Shengzhuang Chen, and Jonathan Richard Schwarz. Scales++: Compute efficient evaluation subset selection with cognitive scales embeddings. *arXiv preprint arXiv:2510.26384*, 2025.
- Raghavendra Chalapathy and Sanjay Chawla. Deep learning for anomaly detection: A survey. *arXiv preprint*, 2019.
- Abhimanyu Das, Weihao Kong, Rajat Sen, and Yichen Zhou. A decoder-only foundation model for time-series forecasting. In *ICML*, 2024. arXiv:2310.10688.
- Hoang Anh Dau et al. The ucr time series classification archive. *IEEE/CAA Journal of Automatica Sinica*, 2019.
- DeepSeek-AI. DeepSeek-V3.1 technical report, 2025. DeepSeek-AI technical report.
- Simon Frieder, Luca Pinchetti, Alexis Chevalier, Ryan-Rhys Griffiths, Tommaso Salvatori, Thomas Lukasiewicz, Philipp Petersen, and Julius Berner. Mathematical capabilities of ChatGPT. In *Advances in Neural Information Processing Systems (NeurIPS), Track on Datasets and Benchmarks*, 2023.
- Clive W. J. Granger. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 1969.
- Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. Large language models are zero-shot time series forecasters. *NeurIPS*, 2023.
- Dan Hendrycks et al. Measuring massive multitask language understanding (mmlu). *ICLR*, 2021.
- Ming Jin et al. Time-llm: Time series forecasting by reprogramming large language models. *ICLR*, 2024.
- Zhijing Jin, Yuen Chen, Felix Leeb, Luigi Gresele, Ojasv Kamal, Zhiheng Lyu, Kevin Blin, Fernando Gonzalez Adauro, Max Kleiman-Weiner, Mrinmaya Sachan, and Bernhard Schölkopf. CLadder: A benchmark to assess causal reasoning capabilities of language models. In *NeurIPS*, 2023.
- Nikolay Laptev, Saeed Amizadeh, and Ian Flint. Generic and scalable framework for automated time-series anomaly detection. *KDD*, 2015.

Alexander Lavin and Subutai Ahmad. Evaluating real-time anomaly detection algorithms – the numenta anomaly benchmark. *IEEE ICMLA*, 2015.

Błażej Leporowski, Daniella Tola, Christian Hansen, and Alexandros Iosifidis. AURSAD: Universal robot screwdriving anomaly detection dataset. *arXiv preprint arXiv:2202.03211*, 2022.

Bryan Lim et al. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *IJF*, 2021.

Mistral AI. Mistral Large 3: Model card, 2025. Mistral AI model card.

Yuqi Nie et al. A time series is worth 64 words: Long-term forecasting with transformers (patchtst). *ICLR*, 2023.

OpenAI. GPT-5-mini: System card and technical report, 2025. OpenAI technical report.

Boris N. Oreshkin et al. N-beats: Neural basis expansion analysis for interpretable time series forecasting. *ICLR*, 2020.

Nick Pawlowski, Daniel C. Castro, and Ben Glocker. Deep structural causal models for tractable counterfactual inference. In *NeurIPS*, 2020.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. *Elements of Causal Inference*. MIT Press, 2017.

Maxime Peyrard and Robert West. A ladder of causal distances. *arXiv preprint arXiv:2005.02480*, 2020.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. *arXiv preprint arXiv:2402.14992*, 2024.

Yujia Qin et al. Tool learning with foundation models. *arXiv preprint*, 2023.

Qwen Team, Alibaba. Qwen-3.5 technical report, 2026. Alibaba technical report.

Donald B. Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology*, 1974.

Lukas Ruff et al. Deep one-class classification. *ICML*, 2018.

Jakob Runge et al. Detecting and quantifying causal associations in large nonlinear time series datasets. *Science Advances*, 2019.

Timo Schick et al. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.

Aarohi Srivastava et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models (big-bench). *TMLR*, 2023.

Ya Su, Youjian Zhao, Chenhao Niu, Rong Liu, Wei Sun, and Dan Pei. Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In *KDD*, 2019.

Ashish Vaswani et al. Attention is all you need. *NeurIPS*, 2017.

Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.

Qingsong Wen et al. Transformers in time series: A survey. *IJCAI*, 2022.

Gerald Woo et al. Unified training of universal time series forecasting transformers. *ICML*, 2024.

Haixu Wu et al. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *NeurIPS*, 2021.

Shunyu Yao et al. React: Synergizing reasoning and acting in language models. *ICLR*, 2023a.

- Shunyu Yao et al. Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*, 2023b.
- Xiang Yue et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. *CVPR*, 2024.
- Ailing Zeng et al. Are transformers effective for time series forecasting? *AAAI*, 2023.
- Guanhua Zhang, Florian E. Dorner, and Moritz Hardt. How benchmark prediction from fewer data misses the mark. *arXiv preprint arXiv:2506.07673*, 2025.
- Haoyi Zhou et al. Informer: Beyond efficient transformer for long sequence time-series forecasting. *AAAI*, 2021.
- Tian Zhou et al. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. *ICML*, 2022.

A Question template inventory

Table 5 lists the number of question templates currently implemented per level, broken down by answer format. Totals reflect the templates shipped in the FactoryBench release at the time of writing and are expected to grow as new templates are added.

Table 5: Number of question templates per level, by answer format.

Level	MC single-select	MC multi-select	Ranking	Tensor / Numerical	Free-form	Total
1 (State)	1	1	1	1	0	4
2 (Intervention)	0	2	1	2	0	5
3 (Counterfactual)	0	2	1	2	0	5
4 (Decision)	0	0	2	0	2	4
Total	1	5	5	5	2	18

Update counts when new templates are merged.

B FactoryBench-Lite: distillation methodology and validation

FactoryBench-Lite appendix: not yet implemented. Add details like:

- **Motivation and prior art:** anchor on miniGHOSTS Frieder et al. [2023] (stratified selection preserving per-file mean/std); contrast with rank-preservation Polo et al. [2024] and cognitive-scales Bean et al. [2025].
- **Selection procedure:** stratify by $(\ell, f, d, \delta) = \text{level} \times \text{format} \times \text{source} \times \text{difficulty}$; within each stratum select $\lceil 0.1 |\mathcal{Q}_{\ell, f, d, \delta}| \rceil$ items minimising $\sum_{m \in \mathcal{M}_{\text{ref}}} (|\mu_m(\mathcal{S}) - \mu_m(\mathcal{Q})| + |\sigma_m(\mathcal{S}) - \sigma_m(\mathcal{Q})|)$; greedy swap from stratified uniform init.
- **Up-front cost:** $|\mathcal{M}_{\text{ref}}| \times C_{\text{full}}$, paid once.
- **Validation:** Spearman ρ / Kendall τ between per-model aggregates on $\mathcal{S}$ vs. $\mathcal{Q}$, plus per-level MAE; acceptance $\rho \geq 0.9$ and $\text{MAE} \leq 3$ pp.
- **Scope:** extrapolation caveat Zhang et al. [2025]; Lite for development/ablations/budget-constrained runs, full benchmark remains authoritative for leaderboard claims.

C Details of the FactoryWave dataset

FactoryWave was collected on two one-armed platforms: the **Universal Robots UR3** (cobot; OnRobot Screwdriver and two-finger gripper variants) and the **KUKA KR10** (industrial arm). Each episode corresponds to one complete task cycle; episodes are recorded under four conditions (normal, fault, trajectory-optimization, and counterfactual), described below. Because state interpretation rather than policy quality is the object of study, motion is generated by a deterministic scripted controller. Every sample carries extensive labelling metadata, and the full native-rate sensor stream is logged without online feature selection.

C.1 Tasks and phase structure

Three canonical tasks are represented in the dataset: Pick-and-Place, Screwing, and Peg-in-Hole. All three are executed on both robots, using matched scripted controllers and task layouts so that cross-embodiment comparisons hold the task fixed. We structure each task as a deterministic sequence of phases (Tables 6, 7, 8); the phase index is written as a discrete label at every timestep, providing an interpretable segmentation that downstream models can condition on or that can be used to localize questions to a specific stage of the cycle. Within this fixed phase skeleton, we randomize as much of the episode as the task permits so that a question template that targets a given phase does not accidentally learn a fixed pose, timing, or trajectory. Pick and place locations are sampled uniformly from predefined workspace regions; phase-specific joint velocities, accelerations, and end-effector rotations are drawn from bounded per-phase ranges; and approach angles to contact events are perturbed within the tolerances of the controller. This forces the Q&A pairs generated from

FactoryWave to reflect phase-level reasoning rather than memorisation of a single scripted trajectory, which in turn makes them generalizable to a wide variety of trajectories and variation found in real recordings.

Table 6: Task phases for Pick-and-Place.

Phase index	Phase name	Description
0	<code>above_pick</code>	EEF moves to a waypoint directly above the object
1	<code>descend</code>	EEF lowers toward the object
2	<code>settle</code>	Brief hold to let physics settle before grasping
3	<code>close</code>	Gripper closes around the object
4	<code>lift</code>	EEF lifts the grasped object off the surface
5	<code>move_xy</code>	Lateral transfer toward the bin
6	<code>lower</code>	EEF lowers into the bin
7	<code>open</code>	Gripper opens, object released
8	<code>retract</code>	EEF moves away from the bin
9	<code>return</code>	EEF returns to home configuration

Table 7: Task phases for Screwing.

Phase index	Phase name	Description
0	<code>approach</code>	EEF moves to a pre-contact waypoint above the fastener
1	<code>descend</code>	EEF lowers to engage the fastener
2	<code>screw</code>	Continuous downward force + wrist rotation to thread
3	<code>disengage</code>	Gripper opens / tool lifts off
4	<code>retract</code>	EEF returns to a safe height
5	<code>descend</code>	EEF lowers to engage the fastener (loosening pass)
6	<code>loosen</code>	Loosen the screw
7	<code>engage</code>	Gripper closes on the retrieved screw
8	<code>retract</code>	EEF returns to home position

Table 8: Task phases for Peg-in-Hole.

Phase index	Phase name	Description
0	<code>above_hole</code>	EEF moves above the hole and aligns
1	<code>insert</code>	EEF pushes peg downward into the hole
2	<code>disengage</code>	Gripper opens
3	<code>above_hole</code>	EEF rises back to the waypoint above the hole
4	<code>retract</code>	EEF returns to home position
5	<code>above_object</code>	EEF moves above the object and aligns with it
6	<code>engage</code>	Gripper closes
7	<code>above_object</code>	EEF rises back to the waypoint above the object
8	<code>retract</code>	EEF returns to home position

C.2 Episode metadata

Alongside the sensor stream, every episode carries a fixed set of metadata fields describing the robot, task, condition, payload, and end-effector configuration (Table 9). These fields are constant within an episode and are used both as filters during question generation (e.g. to restrict a template to a specific payload regime) and as provenance in the released Q&A pairs. This includes fault-specific fields (such as injection timestamp or physical offset).

C.3 Fault taxonomy

FactoryWave includes 28 task-specific fault types together with 2 task-independent software faults, for a total of 30 distinct root causes (Table 10). Each fault is paired with a plain-language description of the underlying physical or control-level mechanism and the injection procedure used to induce it during data collection. The last three columns indicate which tasks include the fault in the

Table 9: Default per-episode metadata fields.

Field	Description
episode_id	Unique identifier for the episode
robot_model	Robot model (UR3, KUKA KR10)
task	Task (pick-and-place, screwing, peg-in-hole)
condition	Episode condition (normal, fault, optimization, trajectory-opt, counterfactual)
fault_id	Fault index for fault episodes; null otherwise
weight_of_box	Mass of the transported object in kg (pick-and-place only)
shape_of_box	Shape/dimensions of the transported object (pick-and-place only)
position_of_box	Initial XYZ position of the object on the table, where measurable
gripper_model	Gripper model and type (vacuum, two-finger, screwdriver)
payload_mass_configured	Payload mass configured at the controller (kg)
payload_cog_configured	Payload CoG offset configured at the controller (x, y, z in mm)
tcp_offset_configured	TCP position offset configured at the controller (x, y, z in mm)
tcp_orientation_configured	TCP orientation configured at the controller (rx, ry, rz in radians)

collected dataset; faults that apply to all three tasks (e.g. collisions with a shared workspace object, misconfigurations of the shared controller) are recorded under each applicable task with task-specific signal imprints.

Table 10: Fault catalogue. Columns *PP*, *Scr*, *PiH* indicate whether the fault is included in the Pick-and-Place, Screwing, and Peg-in-Hole recordings.

Fault		Explanation	Injection	PP	Scr	PiH
Damaged thread	screw	Screw thread physically damaged, preventing proper engagement during tightening.	Pre-damaged the screw thread with sandpaper.	×	✓	×
Missing screw		Tightening is attempted with no screw present.	Removed the screw from screwdriver before the cycle.	×	✓	×
Damaged plate thread		The threaded hole in the plate is damaged, preventing engagement.	Pre-damaged the plate hole with a metal screwdriver.	×	✓	×
Loosening phase		A counterclockwise loosening rotation replaces tightening; a normal phase with a distinct signal. It is counted here as an anomaly to match the fault schema of the AUR-SAD dataset Leporowski et al. [2022].	Reversed the rotation direction in the controller program to execute a counterclockwise loosening pass in place of tightening.	×	✓	×
Gripper activation failure		The vacuum gripper fails to activate and never picks up the box.	Disabled the gripper activation command in the robot program so the pick cycle completes without a grasp.	✓	×	×
Gripper release during motion		The gripper releases the payload mid-trajectory, causing an abrupt payload loss.	Triggered a programmatic gripper-release command at a scripted timestep mid-trajectory.	✓	×	×
Additional axis payload		A dead weight attached to one link increases inertia and gravity loading on all joints.	Bolted calibrated weights of varying mass to one robot link.	✓	✓	×

Table 10 (cont.)

Fault	Explanation	Injection	PP	Scr	PiH
Collision with foam object	Contact with a soft foam block produces a brief TCP force spike without a protective stop.	Placed a foam cube directly in the programmed TCP trajectory.	✓	✓	✓
Unexpected payload weight	The transported box has a weight that deviates from the nominal payload configuration.	Swapped the nominal box for a known heavier or lighter one. This is done in a counterfactual context, ie. switching weights and asking about alternate weight scenario.	✓	×	×
Invalid gripping position	Timing error: the gripper closes after the arm has begun lifting, gripping the object off-centre.	Added a <code>sleep()</code> call in the controller program so gripper closure lags the lift motion.	✓	×	×
Unstable mounting platform	Base instability introduces low-frequency vibrations into the arm.	Placed 8 layers of towels under the base plate.	✓	×	×
Joint position limit violation	A joint moves outside its configured soft or hard position limits, triggering a safety stop.	Random waypoint slightly beyond the configured soft joint limit.	✓	×	×
TCP frame misconfiguration	TCP frame or mounting orientation misconfigured at the controller; gravity torques deviate.	Entered an incorrect TCP offset or mounting angle in the controller installation settings.	✓	✓	✓
Payload weight misconfiguration	Payload mass misconfigured while a tool or workpiece is physically attached.	Left the configured payload weight at multiple wrong mass configurations while a task-dependant gripper remained mounted.	✓	✓	×
External arm disturbance	A continuous external force pulls or pushes the TCP during motion.	Anchored an elastic rope between the bench frame and the tool flange.	✓	✓	✓
Mild payload CoG misconfiguration	Payload CoG specified at an incorrect offset from the tool flange; gravity compensation is wrong.	Entered a CoG offset in the payload configuration that differs from the physical tool CoG.	✓	✓	✓
Collision with hanging cable	A loose cable drags along a robot link or the tool flange during motion.	Suspended a loose cable across the trajectory at arm height.	✓	✓	✓
Collision with cardboard object	A cardboard carton in the trajectory provides moderate resistance; may trigger a protective stop.	Placed a free-standing or braced cardboard box in the programmed trajectory.	✓	✓	✓
Collision with rigid object	A rigid plastic block in the TCP path triggers an immediate protective stop.	Clamped a rigid plastic block in the programmed TCP path. Contrarily to the other collisions, this fault consistently causes safety stops.	✓	✓	✓
Peg insertion misalignment	Peg approaches the hole at an angular or lateral offset and jams against the rim.	Added a recorded lateral offset at the insertion approach waypoint.	×	×	✓

Table 10 (cont.)

Fault	Explanation	Injection	PP	Scr	PiH
Hole obstruction	Foreign object or debris in the hole prevents full insertion.	Placed a small object (metal screw) inside the hole before insertion.	×	×	✓
Incorrect insertion depth	Insertion terminates at an incorrect Z depth due to a mis-configured waypoint.	Shifted the insertion endpoint waypoint Z by recorded offset from nominal.	×	×	✓
Peg surface contamination	Contamination or roughness on the peg raises insertion friction and produces stick-slip.	Applied dry chalk powder to the peg surface.	×	×	✓
Fixture displacement	Insertion fixture shifted laterally, breaking the match between programmed approach and actual hole.	Physically shifted the hole fixture by recorded offset without updating the robot program.	×	×	✓
Self-collision	Arm configuration causes a link to contact the robot body or mounting fixture, triggering a protective stop.	Random waypoint that forces a self-colliding configuration or offset the mounting fixture into the trajectory.	✓	×	×
Missing box	Box absent from the pick position; the gripper closes on empty air.	Removed the box from the pick position before the episode started.	✓	×	×
Missing peg	Arm descends into the hole empty-handed; insertion produces no contact force.	Removed the peg from the gripper before the episode started.	×	×	✓

C.4 Trajectory-optimization episodes

Trajectory-optimization episodes capture the variability of executions that all complete the same task goal. Rather than following the fixed scripted path, the controller is driven between fixed waypoints using randomly sampled intermediate waypoint sequences, so that approach angles, intermediate poses, joint configurations, and motion speeds vary across episodes. This randomized experiment provide a proper source of comparisons for Level 4 optimization-centered templates.

C.5 Counterfactual episodes

Counterfactual pairs are constructed for the subset of faults in Table 10 whose injection moment can be controlled at a specific timestep rather than being present throughout the episode. For each such fault, the initial conditions of the physical setup (object identity and pose, payload configuration, controller settings) are held as constant as the platform allows, one baseline execution is recorded without any injection, and several fault executions are then recorded under the same initial conditions with the fault applied at a shared sampled timestep. Because the physical system is not perfectly reproducible, the pre-injection sensor trajectories of the baseline and the fault candidates diverge slightly; we retain, as the counterfactual partner of the baseline, the fault candidate whose pre-injection window is closest to the baseline in KL divergence, and discard the others. This yields an approximate counterfactual pair in which the early history is as matched as the real platform permits, while the post-injection divergence isolates the effect of the intervention. The approximation error introduced by this procedure, relative to the exact counterfactuals available in simulation, motivates the sim2real analysis in Section 5.3. Each retained pair stores four additional fields, `cf_fault_id`, `cf_injection_timestep`, `cf_injection_time_s`, and `cf_baseline_episode_id`, so that the baseline and fault episodes can be reassembled at analysis time.

C.6 Signal schema

The UR3 is recorded through the `ur_rtde` Python interface at 125 Hz, yielding 137 per-sample signals grouped by semantic role (Table 11): controller *setpoints*, actual *effort* / *feedback* readings from joints and the TCP, *context* / *health* signals covering joint temperatures, voltages, and internal controller state, a small block of controller *status* / *mode* indicators, and digital/analog I/O lines. Six episode-level provenance columns (`episode_id`, `ctx_robot_type`, `ctx_task`, `ctx_fault_label`, `ctx_payload_kg`, `ctx_recording_date`) are appended per episode but are constant within an episode.

Table 11: UR3 signal groups recorded at 125 Hz via `ur_rtde` (excluding episode-level provenance columns).

Group	# signals	Contents
Setpoint (intent)	42	Target joint position, velocity, acceleration, current, torque; target TCP pose and speed.
Effort / feedback	48	Actual joint position, velocity, current, current-as-torque, joint control output; actual TCP pose, speed, and force/torque.
Context / health	33	Joint temperatures, joint voltages, joint modes, tool accelerometer, raw F/T wrench, main/robot voltage and current, momentum, speed scaling, execution time.
Status / mode	7	Timestamp, robot mode, robot status, safety mode, safety status bits, runtime state, configured payload mass.
I/O	6	Digital inputs/outputs, two analog inputs, two analog outputs.
Total	137	

The KUKA KR10 is controlled through the KRC4 controller. We stream 44 per-sample signals at 83 Hz, grouped by semantic role in Table 12 using the same partition as the UR3 schema. A handful of signals that the UR3 exposes natively are unavailable on KSS 8.3, specifically joint velocities, commanded TCP pose, joint voltage, the tool-mounted accelerometer, and joint-level control outputs; TCP force/torque is also absent because no force sensor is fitted.

Table 12: KUKA KR10 signal groups recorded at 50 Hz via RSI/KRL (excluding episode-level provenance columns).

Group	# signals	Contents
Setpoint (intent)	6	Commanded joint positions (degrees).
Effort / feedback	24	Actual joint positions (degrees), actual TCP pose (mm / degrees), per-axis motor current (% of max), and per-axis motor torque (Nm).
Context / health	11	Per-axis motor temperature (Kelvin), Cartesian acceleration on X, Y, Z and absolute magnitude (m/s^2), and speed override (%).
Status / mode	1	Controller process state (FREE / ACTIVE / STOP / END).
I/O	2	Digital inputs bitmask and digital outputs bitmask.
Total	44	

D Example question-answer pairs and question templates

Add a one-page table showing representative Q&A pairs sampled from the dataset (one example per level and per answer format), alongside the underlying question templates they were instantiated from. Include: rendered prompt, ground-truth answer, answer format, originating template, and dataset provenance.

E Simulation setup

After Karim finishes the simulations, we fill this section up. Describe the simulation pipeline used for sim2real analysis: physics engine and version, robot URDFs, controller stack, supported tasks and fault applicators, randomization strategy, logging frequency and signal coverage, and how simulated episodes are aligned with the FactoryWave schema.